

# Raspberry Pi 上の Embedded Xinu のマルチコア化

— アクター言語処理系のための並列実行方式の設計と実測 —

児玉 靖司

法政大学 データサイエンスセンター

2026 年 9 月 10 日

## 概要

教育用小型オペレーティングシステム Embedded Xinu を Raspberry Pi 3 / 4 / 5 の 3 世代の実機上でマルチコア化し、その上でアクター指向言語 AIPL の処理系を並列実行する基盤を実装した。並列実行方式として、核 0 が各二次コアの郵便箱へ仕事を静的に配る **ワーカーズ型**と、各コアが共有 ready キューから自分で取る **対称 SMP 型**の 2 つを同一カーネルに実装し、実行時に切り替えられるようにして比較した。

主要な結果は次の 4 点である。(1) ワーカーズ型については、静的分割の makespan モデルが実測を 5% 前後で説明し、**Cortex-A76 と A72 で比の曲線がほぼ一致する**。(2) 静的分割の弱点は「静的であること」ではなく**分割の粗さ**であり、重い仕事が散らばっていれば分割数を増やすだけで理想に近づく。(3) 仕事を完全に等量にしても 4 コアで 4 倍は出ず、2→4 コアで 1 反復の所要が A76 で +27.7%、A72 で +18.4% 伸びる。データキャッシュを無効にして走らせている構成の帰結であり、N-Queens が 2.2× で頭打ちになる主因である。(4) 対称 SMP 型は AIPL を正しく走らせられなかった。原因は**配布側の呼び手が自分の持ち分を計算すると、先取りのたびにタイマ 1 ティックを失う**ことにあり、399 回の先取り  $\times 10.4 \text{ ms} = 4.15 \text{ 秒}$ が実測と一致した。

さらに、AArch64 への再実装過程で**例外復帰時に ELR\_EL1 / SPSR\_EL1 を退避していない**という重大な欠陥を Pi 5 と Pi 4 の双方で発見・修正した。先取りを有効にすると、8 プロセス中 8 個すべてが誤った計算結果を返す。同じ機能を持つ ARM32 版 (Pi 3) は当該状態をスレッド自身のスタックに退避しており、この欠陥は **64 ビット化の際に落としたもの**であった。本稿では、こうした**実装上の落とし穴と測定手続き自体の検証法**を結果と同じ重みで報告する。

## 1 はじめに

### 1.1 背景

Embedded Xinu[1, 2] は、教育用に設計された小型のオペレーティングシステムである。構造が単純で全体を読み切れることから、オペレーティングシステム教育および組込みシステムの研究基盤として広く用いられてきた。近年これを Raspberry Pi へ移植する試みが行われているが、**その多くは単一コアを前提としている**。Raspberry Pi 3 / 4 / 5 はいずれも 4 コアの ARM プロセッサを搭載しており、計算資源の 4 分の 3 が使われないまま残っている。

一方、本研究室ではアクター指向言語 AIPL とその処理系を開発している。アクターモデル [3] に基づく言語は、メッセージ送受の単位が細かく、1 つのメソッド本体の実行時間が数十マイクロ秒に満たないことが珍しくない。このような処理系を複数コアで動かすとき、「**メッセージをどう配るか**」の設計がそのまま性能を決める。

## 1.2 問題設定

マルチコア化には、大きく 2 つの設計がある。

**ワーカーズ型（郵便箱・静的分割）** 核 0 が仕事を区間に分け、各二次コアの郵便箱へ書き込んで起こす。仕事は**データ**として配られ、1 コアに 1 区間が固定される。

**対称 SMP 型（共有 ready キュー）** 仕事を通常のプロセスとして生成し、全コアが共有 ready キューから自分で取る。仕事は**プロセス**として配られる。

前者は汎用性に欠けるが軽い。後者は汎用的だが、プロセス生成・キュー操作・文脈切替・先取りの費用を、寿命の短い仕事にも払うことになる。**AIPL** のようにメッセージが短い処理系では前者が有利であるというのが本研究の作業仮説である。

## 1.3 本稿の貢献

1. Raspberry Pi 3 / 4 / 5 の **3 世代の実機**で Embedded Xinu をマルチコア化し、同一のベンチマーク基盤（N-Queens・素数計数・食事する哲学者・メモリ充填・AIPL アクター並列）を実装した（第 3 節）。
2. 上記 2 方式を同一カーネルに実装し、実行時に切り替えられるようにした。同じ起動・同じ温度条件のまま A/B を取れるため、方式以外の要因を排除できる（第 3 節）。
3. AArch64 への再実装で共通して落とされていた**例外復帰の欠陥**を発見し、同一起動内での A/B により実証したうえで修正した（第 4 節）。
4. **測定手続き自体を検証する方法**を確立した。計器を先に作り、計器が嘘をつかないことを別の計器で確かめ、正誤の検算を測定値と同時に採る（第 5 節）。本稿では、この方法によって**自分の初期測定に見つけた 3 つの誤り**も報告する。
5. 静的分割の弱点が「静的であること」ではなく「分割の粗さ」であることを、2 つのアーキテクチャで示した（第 6 節）。
6. アクターを**データ**として扱う方式と**スレッド**として扱う方式の違いが、速度だけでなく**系全体の可用性**に及ぼす影響を実測した（第 6 節）。

## 2 対象と前提

### 2.1 3 世代の実機

|          | Raspberry Pi 3 B+     | Raspberry Pi 4 | Raspberry Pi 5  |
|----------|-----------------------|----------------|-----------------|
| CPU      | Cortex-A53 × 4        | Cortex-A72 × 4 | Cortex-A76 × 4  |
| 実行モード    | <b>ARM32 (ARMv7)</b>  | AArch64        | AArch64         |
| カーネル     | kernel7.img           | kernel8.img    | kernel_2712.img |
| 系統       | classic Embedded Xinu | AArch64 再実装    | AArch64 再実装     |
| データキャッシュ | 有効                    | <b>無効</b>      | <b>無効</b>       |
| 更新手段     | SD 交換                 | SD 交換          | USB 交換          |

Pi 3 は classic Embedded Xinu の系統を保った 32 ビット実装であり、Pi 4 / Pi 5 は AArch64 で書き直された別系統である。この**系統の違いが、後述する欠陥の有無をそのまま決めている**。

3 板ともデータキャッシュの扱いが単純ではない。Pi 4 / Pi 5 では、実験の再現性を優先してデータキャッシュを**無効**にした状態で走らせている。この選択は 2 つの帰結をもたらす。第一に、非キャッシュ可能メモリ上では `ldaxr/stxr` による排他が保証されないため、**原子命令を使わない排他制御**が必要になる。第二に、すべてのメモリ参照がバスに出るため、**コアを増やすほど 1 命令あたりの所要が伸びる** (第 6 節)。

## 2.2 AIPL とその実行基盤

AIPL はアクター指向の言語で、効果注釈・期限・第一級の返信先などを持つ。本稿で用いるのは、AIPL をバイトコード (AVM) へコンパイルし、カーネル内の仮想機械で実行する経路である。

Pi 4 / Pi 5 では、アクターは**データ**である。オブジェクト表 `vm_obj[]` とメッセージ待ち行列 `vm_q[]` を持ち、仮想機械が待ち行列から取り出してメソッド本体を実行する。Pi 3 では、アクターは **Xinu スレッド**である (アクターごとに `tid_typ` とセマフォによるメールボックスを持つ)。バイトコード仮想機械はそのスレッド機構の上に載っている。**この違いは移植の都合ではなく、方式そのものの違い**であり、第 6 節で比較の対象とする。

## 3 マルチコア化の設計

### 3.1 二次コアの起動と排他制御

二次コアは起動後スピンし、郵便箱に仕事があるまで `wfe` で待つ。共有データの排他には、当初 ARM の排他命令 (`ldaxr/stxr`) を用いた。しかしデータキャッシュ無効の構成では**排他命令の動作が保証されない**。実機では「Xinu の起動画面が出た直後に停止する」という形で現れた。

そこで **Lamport のパン屋のアルゴリズム** [5] による、原子命令を使わない排他に置き換えた。ビルド時に `SPIN=bakery` と `SPIN=exclusive` を選べるようにし、既定を前者とした。排他機構そのものが正しいことは、4 コアが各 20,000 回加算して 80,000 になることを確かめる自己診断口 (`/smplock`) で毎回検証する。

### 3.2 方式 A: ワーカーズ型

核 0 が区間  $[0, n)$  を「コア数」個に分け、各二次コアの郵便箱 (関数ポインタ・区間・世代番号) へ書き込み、`dsb; sev` で起こす。核 0 自身も 1 区間を担当する。完了は世代番号の一致で判定する。

区間の切り方には注意を要する。 $chunk = n / ncores$  の整数除算で余りを最後のコアに載せると、 $n$  がコア数で割り切れないときにその 1 コアだけが余りを抱え、`makespan` が跳ねる。 $lo = cn / nc$ 、 $hi = (c+1)n / nc$  とすれば余りは各コアへ 1 つずつ散る。

### 3.3 方式 B: 対称 SMP 型

仕事を通常の Xinu プロセスとして生成し、共有 ready キューへ載せる。各コアは自分の idle ループでキューから取り出して実行する。キューはスピンロックで保護し、**文脈切替を跨いでロックを保**

持し、再開した側が離す受け渡し方式とする。こうしないと、「ready に戻したがスタックポインタをまだ書いていないプロセス」を他コアが拾ってしまう。

### 3.4 同一カーネルでの切り替え

2 方式は同じカーネルに同居させ、/smpmode?on=1 で実行時に切り替える。起動時は必ずワーカーズ型で立ち上がるため、対称側が壊れていても板は起動する — これは実験の初期に、対称スケジューラを起動経路に置いたために板が起動しなくなった経験から得た設計である。

### 3.5 AIPL アクター・バッチの並列配布

仮想機械の待ち行列の先頭から、宛先アクターが互いに異なり、かつ描画命令と WAIT を含まないクラスのメッセージを連ねてバッチとし、そのバッチを上記いずれかの方式で配る。描画・WAIT を含むクラスを除外するのは、それらが大域状態（フレームバッファ・フレーム境界）を触るためで、クラス単位の静的解析（オペコード列をオペランドを読み飛ばしながら走査する）で判定する。

メソッド本体が生むものは 3 つあり、それぞれ分離する。

| 生むもの    | 分離の方法                                   |
|---------|-----------------------------------------|
| メッセージ送信 | コアごとの控え par_q[core] に積み、join 後に本待ち行列へ併合 |
| アクター生成  | オブジェクト表をコアごとに重ならない範囲へ分割                 |
| 描画      | 除外（該当クラスは 1 コア経路）                       |

これにより、バッチ内の各メッセージは自分のアクターの領域と自前の値スタックしか触らないため、錠を用いずに並列実行できる。

## 4 実装上の落とし穴

本節では、マルチコア化の過程で遭遇した欠陥を報告する。いずれも単一コアでは顕在化しないか、特定の使い方でのみ顕在化する性質を持ち、発見には計器の設計が必要だった。

### 4.1 例外復帰時に ELR/SPSR を退避していない

■**症状**。 Pi 5 で N-Queens の並列解が直列解と一致しない。1 タスクなら常に正しく、2 タスク以上で狂う。

■**原因**。 割り込みハンドラ irq\_dispatch\_c は proc\_preempt() → proc\_resched() → ctxsw() を呼び、割り込み区間の中で別プロセスへ切り替わる。ところが例外エントリは汎用レジスタ（および Pi 4 では浮動小数点レジスタ）を退避する一方、ELR\_EL1 と SPSR\_EL1 を退避していなかった。これらはコアごとに 1 組しかないため、割り込まれたプロセス A が待たされている間に別のプロセス B が同じように割り込まれると上書きされ、A は B の復帰先へ eret して戻る。

興味深いことに、Pi 4 の当該箇所には「irq\_dispatch\_c は ctxsw を呼びうるので、割り込まれた側の浮動小数点状態は生き延びねばならない」という注釈が書かれていた。まったく同じ理屈が ELR/SPSR にも当てはまるのに、そこだけ抜けていた。

表 1: Pi 4 における ELR/SPSR 退避の有無（先取り有効・同一起動・期待値 480003）

| プロセス数 | 退避なし（旧挙動）                                | 退避あり（修正）                |
|-------|------------------------------------------|-------------------------|
| 2     | 991055 906373 → <b>2/2 誤り</b>            | 全て 480003 → <b>誤り 0</b> |
| 4     | 428433 258890 504716 480003 → <b>3/4</b> | 全て 480003 → <b>0</b>    |
| 8     | 8 個すべて相異なる値 → <b>8/8</b>                 | 全て 480003 → <b>0</b>    |

■ARM32 版は正しかった。 Pi 3 の ARM32 実装は次のようになっている。

```
sub lr, lr, #4
srsdb #ARM_MODE_SYS! /* SPSR_irq と LR_irq を SYS スタックへ積む */
cpsid if, #ARM_MODE_SYS /* = 割り込まれたスレッド自身のスタック */
push {r0-r4, r12, lr}
bl dispatch /* この中で clkhandler -> resched() が起こりうる */
pop {r0-r4, r12, lr}
rfeia sp! /* 自分のスタックから PC と CPSR を戻す */
```

復帰状態を割り込まれたスレッド自身のスタックに積んでいるため、dispatch() の中で文脈が切り替わっても各スレッドは自分の復帰状態を持って戻る。AArch64 版は、ARM32 版が正しく行っていたことを再実装の際に落としていたのである。

■実証。 修正の効果は、同一起動内で切り替えられる形にして示した。例外復帰時の書き戻しを実行時スイッチ（/elrfix?on=0|1）で入切できるようにし、答えの決まった計算（積・和・剰余のみでメモリを持たない）を複数プロセスで同時に走らせて基準値と突き合わせる計器（/preempttest）を用意した。結果を表 1 に示す。

■計器そのものを先に検証すること。 最初の測定では誤りが 0 であり、欠陥が無いように見えた。しかしそれは Pi 4 が先取りを既定で無効にしているためで、**危険が起こり得ない状態を測っていた**のである。先取りの発火回数を数える計器で fired=0 を確認して初めて、この測定が無意味であると分かった。先取りを有効にすると fired=605 となり、表 1 の結果が得られた。計器が「異常なし」と言うとき、それが「異常が無い」のか「異常が起こり得ない条件で測っている」のかを、別の計器で確かめる必要がある。

## 4.2 核 0 がスケジューラに参加しない

対称 SMP 型で N-Queens を解かせると、コア別の実行数の先頭が常に 0 であった。すなわち核 0 は 1 つもタスクを走らせておらず、実質 3 コアで解いていた。

原因の特定には 2 度の誤りを経た。第一に、核 0 は窓系の描画ループに居ると考えてそこへ引き取り口を置いたが、この構成では HDMI が無く当該関数の呼び出し元が存在しなかった。第二に、核 0 の暇はシリアル入力待ちの空回りだと考えて移したが、シリアルアダプタが送信を受信へ折り返す配線であったため受信 FIFO がほとんど空にならず、引き取り口はほぼ回らなかった。

正解は proc\_preempt() であった。ここは 100 Hz のタイマ割り込みから EOI 後に必ず呼ばれる。しかも既存の if (!IS\_IDLE(currpid)) proc\_resched(); という判定こそが「核 0 は idle だから何もしない」＝不参加の原因そのものであった。修正後、コア別の実行数は 1/1/1/1 となり、N-Queens は 496 ms から 397 ms へ改善した。2 方式の差の約半分は、方式ではなくこの実装の穴が

表 2: 対称 SMP 型における 1 バッチの時間の内訳 (Aip1Eq4)

| 計器                 | 値                | 読み                         |
|--------------------|------------------|----------------------------|
| 待ちループ wait_us      | 1,850            | 生成した単位は <b>1.85 ms</b> で完了 |
| 呼び手の持ち分 inline_us  | <b>4,150,086</b> | ここだけが <b>4.15 秒</b>        |
| 呼び手への先取り pp_inline | <b>399</b>       | その間に 399 回 先取りされた          |
| スタック再利用 reuse      | 0                | 第 4.3 節の危険は解消済み            |

作っていたことになる。

### 4.3 静的スタックの再利用

対称 SMP 型の並列配布では、単位ごとにプロセスを生成する。そのスタックは静的配列を再利用していた。ところが単位プロセスは

```
sp_done++; /* 駆動側はこの瞬間に「終わった」と判断して戻る */
spin_unlock(&proc_sched_lock);
proc_exit(); /* まだ自分のスタックの上で実行中 */
```

という順で終わるため、完了を通知したあとも自分のスタック上で動いている。駆動側は直ちに帰り、次の呼び出しが同じ静的スタックへ新しい文脈を書き込む。

AIPL は毎秒数百回この経路を通るため必ず踏み、N-Queens は要求ごとに 1 回しか通らないため踏まない。スタックを世代で回して再利用まで数バッチ挟むと、誤答は 28~78 件から 0~1 件へ、コア別実行数の偏りは解消した。再利用時に前の占有者がまだ解放されていない回数を数える計器で、この危険が実際に起きていることを数で確認した。

### 4.4 配布側の呼び手が自分で計算すること

上記を直しても、対称 SMP 型では 1 バッチに 4.15 秒（ワーカーズ型の約 20 倍）かかる場合が残った。時間の行き先を分解する計器を入れて、原因が特定できた（表 2）。

$399 \times 10.4 \text{ ms}$ （タイマ 1 ティック）= 4.15 秒であり、inline\_us と一致する。すなわち先取り 1 回につき丸ごと 1 ティックを失っている。

並列配布そのものは健全であった（生成した単位は 1.85 ms で完了）。壊れていたのは呼び手が自分の持ち分を計算するという設計である。ワーカーズ型では呼び手は核 0 に固定され先取りされないため、「呼び手も 1 単位ぶん働く」という自然で効率的な設計が成立する。対称 SMP 型では呼び手は通常のプロセスとして共有 ready キューに載るため、同じ設計が 20 倍の劣化を招く。

これは方式の差が実装の細部に現れた例である。静的分割で自然な設計が、動的スケジューラの上では成り立たない。

## 5 測定の作法

実機の測定では、どの計器を信じてよいかを確かめる過程そのものが結果の一部である。本節では採用した作法と、それによって発見した自分の初期測定の誤りを述べる。

## 5.1 採用した作法

1. 計器を先に作る。板が停止すると何も分からない。焼き直し 1 回は当てずっぽう 5 回より安い。
2. 計器自体を検証する。排他機構は 4 コア × 20,000 回の加算で、並列配布はコア別実行数で、先取りは発火回数で確かめる。
3. 正誤を測定値と同時に採る。N-Queens は毎回 solutions=73712 と直列解との一致を突き合わせる。
4. 同一起動内で A/B を取る。方式・修正の有無を実行時スイッチで切り替え、温度・配置・断片化などの条件を揃える。
5. 暖機を捨て、中央値・最小・最大で報告する。

## 5.2 自分の測定に見つけた 3 つの誤り

上記のうち (3) と (5) は、当初 N-Queens 側にのみ適用しており、AIPL 側では抜けていた。その結果 3 つの誤りが生じた。

■誤り 1: 正誤を確かめていなかった。AIPL の標本は各ラウンドで合計を照合し、正なら色 2・誤なら色 4 の線を引くが、その色は画面にしか出ない。この板には表示装置が無い。標本を書き換えずに済ませるため、カーネル側で線の色を数えることにした (LINE 命令で色 2 なら ok++, 色 4 なら ng++)。ok>0 かつ ng==0 でなければ採用しないという規則を導入した。

■誤り 2: 標本が 2 ラウンドで停止していた。仮想機械への最初のメッセージは読み込み時に一度だけ積まれ、WAIT はフレーム境界を立てるだけで積み直さない。仮想機械を進める関数を呼ぶのは描画ループだけだが、表示装置が無いためそれが呼ばれない。つまり読み込み直後の 2 ラウンドしか走っていなかった。測定窓を明示的に指定する駆動口 (/avm-run?ms=N) を設け、待ち行列が空なら次のラウンドを積み直すようにした。

なお本稿の初期報告にあった「仮想機械のバッチ化が 32 アクターで破綻している」という記述は、この誤りに由来する。M = 32 が特別だったのではなく、どの標本も 2 ラウンドで止まっていたのである。

■誤り 3: その 2 ラウンドは冷間始動の過渡だった。駆動口を入れて回すと、1 バッチの所要時間はバッチ数とともに増えて収束する (93.1 → 148.9 → 177.1 → 183.7 → 定常 186.0 ms)。定常値は測定窓を 2 倍にしても 0.05% 以内で一致する。初期測定 of 絶対値は、すべて定常値の約半分であった。過渡が定常のちょうど半分になる機序は特定できていない。

## 6 評価

以下、AIPL の測定はすべて暖機を捨てた定常値であり、全点で誤答 0 (各ラウンドの合計が検算に通った) を確認してある。

表 3: N-Queens  $n = 13$  の速度向上（ワーカーズ型）

| コア | Pi 5 (A76)              | Pi 4 (A72)              | Pi 3 (A53)              |
|----|-------------------------|-------------------------|-------------------------|
| 1  | 660.1 ms (1.00×)        | 1401.3 ms (1.00×)       | 1426.5 ms (1.00×)       |
| 2  | 355.8 ms (1.85×)        | 959.3 ms (1.46×)        | 779.0 ms (1.83×)        |
| 3  | 306.0 ms (2.16×)        | 859.0 ms (1.63×)        | 560.3 ms (2.55×)        |
| 4  | <b>298.8 ms (2.21×)</b> | <b>847.9 ms (1.65×)</b> | <b>487.1 ms (2.93×)</b> |

## 6.1 N-Queens ( $n = 13$ )

探索木の大きさが列ごとに大きく異なる、不揃いな負荷である。各 3 回の中央値、解は毎回 73,712 で直列解と一致。

Pi 3 と Pi 4 は**直列がほぼ同じ**（1426.5 対 1401.3 ms、差 1.8%）であるのに、4 コアでは Pi 3 が 1.7 倍速い。素直に読めば「A53 のほうが並列に強い」となるが、**これは誤りである**。次節がそれを示す。

## 6.2 均一な負荷なら三板とも同じ

分割の不均等がほとんど無い負荷（素数計数、 $n = 300,000$ ）で測り直すと、Pi 4 と Pi 3 の速度向上曲線は 0.99/1.61/2.31/3.01 対 1.00/1.61/2.31/3.01 と**完全に一致**した（Pi 5 も 4 コアで 3.02×）。したがって表 3 の差はハードウェアの差ではなく、**各カーネルが N-Queens をどう分割しているか**の差である。

実際、Pi 4 で分割の仕方だけを変えると次のようになる（各 2 回、再現性あり）。

| 分割                     | 4 コア                    | 速度向上         |
|------------------------|-------------------------|--------------|
| 連続 4 等分                | 847.6 / 847.3 ms        | 1.64×        |
| ストライド（第 1 行）           | 811.1 / 811.1 ms        | 1.72×        |
| ストライド（第 2 行・Pi 3 と同方式） | <b>774.3 / 774.4 ms</b> | <b>1.81×</b> |

スケジューラを一切変えず、分割の仕方を変えるだけで +10% である。

## 6.3 AIPL: ばらつきと makespan

総仕事量を 128,000 反復に固定し、8 アクターへの配分のばらつき比だけを振る。予測は静的分割の makespan（各コアの持ち分の最大値）である。

実測は予測に 5% 前後で追随し、**A76 と A72 で比の曲線がほぼ一致する**。方式の振る舞いはアーキテクチャに依らない。

表 4: ばらつき比と 1 バッチの所要（総仕事量固定・ワーカーズ型・定常値）

| 比   | Pi 5 (A76) |             | Pi 4 (A72) |             | 予測比         |
|-----|------------|-------------|------------|-------------|-------------|
|     | ms/batch   | 比           | ms/batch   | 比           |             |
| 1   | 186.0      | 1.00        | 253.8      | 1.00        | 1.00        |
| 2   | 248.1      | 1.33        | —          | —           | 1.32        |
| 4   | 308.7      | 1.66        | 412.1      | 1.62        | 1.65        |
| 8   | 365.1      | 1.96        | —          | —           | 1.98        |
| 16  | 415.7      | 2.23        | 557.3      | 2.20        | 2.28        |
| 32  | 460.1      | 2.47        | —          | —           | 2.57        |
| 64  | 498.8      | 2.68        | 671.3      | 2.64        | 2.81        |
| 128 | 534.3      | <b>2.87</b> | 715.2      | <b>2.82</b> | <b>3.01</b> |

#### 6.4 AIPL: 静的分割の弱点は「分割の粗さ」

同じ重み集合（1.. $M$  を正規化、合計 128,000 反復）を、**散らした場合**（ビット反転順という決定的な低食い違い順）と**後ろに固めた場合**（単調増加）で比べる。

|      |     | $M = 4$  | $M = 8$  | $M = 16$             |
|------|-----|----------|----------|----------------------|
| Pi 5 | 散らす | 301.5 ms | 255.5 ms | <b>235.3 ms</b> (改善) |
| Pi 5 | 固める | 301.5 ms | 322.6 ms | <b>349.4 ms</b> (悪化) |
| Pi 4 | 散らす | 408.9 ms | 339.1 ms | <b>300.2 ms</b> (改善) |
| Pi 4 | 固める | 410.6 ms | 424.6 ms | <b>434.7 ms</b> (悪化) |

郵便箱は区間を**連続**に等分するため、重い仕事が散らばっていれば  $M$  を増やすほど各区間の合計が平均化されて理想に近づき、固まっていれば同じコアに集まり続けて**悪化する**。両板で同じ向きに分かれた。

したがって処方は「共有 ready キューへ移れ」ではない。多めに分割し、重い仕事を隣り合わせにするな — これは郵便箱方式を保ったまま実行できる。

#### 6.5 AIPL: 4 コアで 4 倍は出ない

仕事を完全に等量（1 通 32,000 反復）にしたまま、バッチ長で使用コア数だけを変える。バッチ長がコア数に満たないと残りのコアは 1 通も実行しないことを利用している（コア別実行数で確認）。

| 使用コア     | Pi 5 (A76)      |                               | Pi 4 (A72)      |                               |
|----------|-----------------|-------------------------------|-----------------|-------------------------------|
|          | ms/batch        | 1 反復                          | ms/batch        | 1 反復                          |
| 2        | 145.3           | 4.54 $\mu$ s                  | 215.3           | 6.73 $\mu$ s                  |
| 3        | 164.1           | 5.13 $\mu$ s                  | 232.8           | 7.28 $\mu$ s                  |
| 4        | 185.5           | <b>5.80 <math>\mu</math>s</b> | 255.0           | <b>7.97 <math>\mu</math>s</b> |
| 2→4 の劣化  | +27.7%          |                               | +18.4%          |                               |
| 実効スループット | 1.57× (理想 2.00) |                               | 1.69× (理想 2.00) |                               |

分割の不均衡はゼロなのに、コアを増やすと 1 反復の所要が伸びる。データキャッシュを無効にしているため、すべてのメモリ参照がバスに出る。A72 のほうが劣化が小さいのは、素の 1 反復が遅いぶんメモリ帯域への要求が緩いためと読める。N-Queens が 4 コアで 2.2× 止まりだった主因はここにある（分割の不均衡ではない）。

## 6.6 アクター＝データ 対 アクター＝スレッド

Pi 3 に AIPL のバッチ配布を移植しようとして、**移植すべきものが無い**と分かった。Pi 3 のアクターは Xinu スレッドであり、「メッセージを束ねてコアへ配る」段階が存在しない。同じ計器と駆動口を Pi 3 にも実装して測ったところ、2 つの構造的な事実が得られた。

■(1) 1 通も他コアへ配られない。コア別実行数は常に  $N/0/0/0$  である — 4 アクターでも 16 アクターでも、すべてのメッセージが核 0 で実行される。アクターをスレッドにしても、それだけでは多コアは使われない。同じ標本で Pi 4 / Pi 5 は  $N/N/N/N$  と均等に散る。

■(2) 計算の重いアクターを 8 個載せると系が止まった。HTTP がまったく応答しなくなった (ping は 100% 通るのでネットワーク層は生存)。原因は仮想機械アクターの優先度が既定値のままで、Web サーバと同値だったことである。優先度を下げて解消した。アクター＝データ方式では仮想機械が協調的に進むため、優先度の設計を誤っても系は止まらない。

■定量比較は保留する。Pi 3 では複数のディスパッチが同時に計時中になり、壁時計の区間が重なって積算されるため (2 秒の窓に 21.4 秒を計上)、1 メッセージあたりの費用は Pi 4 / Pi 5 と比較できない。共通の物差しは検算に通ったラウンド数だが、Pi 3 は暖機後に一定数で停止する未解決の問題があり、スループットを並べるのは時期尚早である。

## 6.7 対称 SMP 型と AIPL の比較は取れなかった

第 4.4 節に述べたとおり、対称 SMP 型は AIPL を正しく走らせられなかった。検算計器を入れると誤答・停止・板の停止が現れる。したがって本稿では、2 方式の AIPL における定量比較を報告しない。

ただし壊れ方に情報がある。同じコードが N-Queens (要求ごとに 1 回、重い区間を配る) では正しく動き、AIPL (毎秒数百回、短いバッチを配る) でのみ壊れる。壊れ方が「短く・頻繁」という条件に選択的であることは、本研究の作業仮説 — 短いメッセージを汎用スケジューラに載せる費用が高い — と方向が一致する。

## 7 考察

### 7.1 何が言えて、何が言えないか

本稿の測定はワークス型については完結している。一方対称 SMP 型との定量比較は取れていない。したがって「2 方式のどちらが速いか」を AIPL について数値で述べることはできない。言えるのは次の 3 点である。

1. 静的分割の makespan モデルは実測をよく説明し、アーキテクチャに依らない。ばらつき比を振ったとき、実測は予測に 5% 前後で追随し、A76 と A72 で比の曲線がほぼ一致する。
2. 静的分割の弱点は「静的であること」ではなく「分割の粗さ」である。重い仕事が散らばっていれば分割数を増やすだけで理想に近づく。この処方では郵便箱方式を保ったまま実行できる。
3. 並列化の天井はメモリにある。仕事を完全に等量にしても 4 コアで 4 倍は出ず、2→4 コアで 1 反復が 18 ~ 28% 伸びる。

### 7.2 仮説を支持する間接的な材料

直接の対決は測れなかったが、次の 3 つはいずれも作業仮説の向きを支持する。

1. 対称 SMP 型はそもそも AIPL を正しく走らせられなかった。原因は呼び手が先取りのたびに 1 ティックを失うことであり、これは短い仕事を汎用スケジューラに載せる費用そのものである。同じコードが N-Queens では正しく動く。
2. アクター=スレッドの板では、1 通も他コアへ配られない。アクターをスレッドにすること自体は多コア利用を意味しない。
3. アクター=スレッドの板では、計算の重いアクターが系を止めた。アクター=データ方式では起こらない。速度以外の面でも堅い。

### 7.3 アーキテクチャ上の含意

アクターモデルの言語処理系では、メッセージはデータとして配るのが安い。プロセスとして配ると、汎用スケジューラが長寿命スレッドを前提に払っている費用（生成・キュー操作・文脈切替・先取り）を、寿命が 100  $\mu$ s に満たない仕事にも払うことになる。本稿で対称 SMP 型が壊れた形 — 呼び手が先取りされて 1 ティックずつ失う — は、その費用が最も露骨に現れた例である。

Raspberry Pi 上の Embedded Xinu で AIPL を実行する基盤としては、バッチ化した郵便箱方式を既定とし、分割数を仕事の不揃いさに応じて増やすのが妥当と考える。動的分配の併用は、第 4.4 節の問題を解いたうえで改めて評価すべきである。

### 7.4 教育用 OS をマルチコア化することの意義

本稿で報告した欠陥は、いずれも単一コアでは顕在化しないか、特定の使い方でのみ顕在化する。例外復帰の欠陥（第 4.1 節）は先取りを有効にした瞬間に全プロセスの計算結果を壊すが、先取りが既定

で無効なら誰も気づかない。静的スタックの再利用（第 4.3 節）は呼び出し頻度が低ければ踏まない。

教育用 OS をマルチコア化する作業は、こうした「単一コアでは見えない前提」を洗い出す作業でもある。本稿がそれらを結果と同じ重みで報告したのは、そのためである。

## 8 関連研究

Embedded Xinu[1, 2] は教育用 OS として広く用いられており、複数のアーキテクチャへの移植が行われている。本研究はそれを Raspberry Pi の 3 世代へ展開し、マルチコア化と、その上でのアクター言語処理系の実行という点で従来と異なる。

アクターモデル [3] に基づく実行系では、アクターを軽量プロセスとして扱う方式が広く採られている (Erlang[4] など)。本稿の Pi 3 実装はこの系統に近い。一方、Pi 4 / Pi 5 実装はアクターをデータとして扱い、メッセージをバッチ化してコアへ配る。同一の言語処理系に対して両方式を実装し、同じ計器で比較した点が本稿の位置づけである。

原子命令を用いない排他制御には Lamport のパン屋のアルゴリズム [5] を用いた。これはデータキャッシュ無効という本構成固有の制約に対する選択である。

AArch64 の例外モデルおよび退避すべき状態については [6] に従う。

**注記:** 本節は骨子のみを示す。AIPL および本研究室の先行研究に関する引用、ならびに組込み OS のマルチコア化に関する個別の先行事例については、投稿先の様式に合わせて補うものとする。

## 9 おわりに

Embedded Xinu を Raspberry Pi 3 / 4 / 5 の 3 世代でマルチコア化し、アクター言語 AIPL の処理系を並列実行する基盤を実装・評価した。

ワーカーズ型については、静的分割の makespan モデルが実測を 5% 前後で説明し、その弱点が「静的であること」ではなく「分割の粗さ」であること、並列化の天井がメモリにあることを、2 つのアーキテクチャで示した。対称 SMP 型については、AIPL を正しく走らせられないことと、その原因が呼び手が先取りのたびにタイマ 1 ティックを失うことにあることを、時間の内訳を分解する計器によって特定した。

また、AArch64 への再実装で共通して落とされていた例外復帰の欠陥を発見・修正し、ARM32 版が同じ状態を正しく扱っていたことを示した。マルチコア化は、単一コアでは見えない前提を洗い出す作業でもある。

今後の課題は、(1) 呼び手に持ち分を持たせない構成での対称 SMP 型の再評価、(2) 郵便箱方式へのワークスティーリングの導入、(3) Pi 3 のスレッド型実行系の多コア利用、(4) データキャッシュを有効にした構成での再測定である。

## 参考文献

- [1] D. E. Comer, *Operating System Design: The XINU Approach*, 2nd ed., CRC Press, 2015.
- [2] Embedded Xinu Project, Marquette University. <https://embedded-xinu.readthedocs.io/>
- [3] G. Agha, *ACTORS: A Model of Concurrent Computation in Distributed Systems*, MIT Press, 1986.

- [4] J. Armstrong, *Making Reliable Distributed Systems in the Presence of Software Errors*, PhD thesis, Royal Institute of Technology, Stockholm, 2003.
- [5] L. Lamport, “A New Solution of Dijkstra’s Concurrent Programming Problem,” *Communications of the ACM*, vol. 17, no. 8, pp. 453–455, 1974.
- [6] Arm Ltd., *Arm Architecture Reference Manual for A-profile architecture*.